

Controle de Estoque

Hospital Odontológico • UFPE

Da planilha de Excel a
um sistema com
rastreadabilidade, validade
e pedidos digitais.

Kaique • Alex • Wallyson • Bruno • Vinícius • José • Luiz



O PONTO DE PARTIDA

Um estoque que vive em planilhas

- ◆ O hospital controla materiais por **planilhas de Excel**.
- ◆ Resultado: divergência entre o estoque **registrado e o físico**, sem rastreabilidade de pedidos.
- ◆ **Perda de material por vencimento**, sem nenhum alerta de validade.
- ◆ Pedidos chegam por **e-mail e papel**, em caixas dispersas.

Público-alvo: gestores, almoxarifes e equipes clínicas do Hospital Odontológico.



— A VIRADA DO PROJETO

O insight que mudou o escopo

Diziam: "o CEO *não tem controle de estoque*". Investigando o fluxo real, vimos que o CEO **não tem estoque próprio** — tudo depende do **HO**, a Dispensação que abastece os setores.

Reposicionamento

- ◆ **HO** = fonte de verdade do estoque.
- ◆ **CEO** = célula-piloto de organização.
- ◆ Arquitetura **modular**: novas células acopláveis depois.

Por que importa

Recebemos a problemática com informação incompleta e lógica equivocada. Recolocá-la corretamente tornou a solução viável e escalável, e nos levou a documentar tudo para quem vier depois.



A SOLUÇÃO EM UMA TELA

O que o sistema entrega

- ◆ **Catálogo × Lotes** (1:N) com validade e segregação.
- ◆ **Pedidos multi-item**, status derivado, aprovação parcial via FEFO.
- ◆ **Estoque do CEO** alimentado automaticamente pelas expedições.
- ◆ **Dashboards por perfil**: crítico, vencendo, demanda represada.
- ◆ Quatro perfis: Gestor HO, Almoxarife, Gestor CEO, Solicitante.
- ◆ **Agente de e-mail** (EP08): uma decisão de UX.



COMO TRABALHAMOS

Gestão e fluxo de desenvolvimento

Método

- ◆ **Scrum/Kanban** com board no Jira.
- ◆ Backlog: épicos → User Stories (Gherkin, MoSCoW).
- ◆ Divisão por pacotes entre os sete membros.

Fluxo

- ◆ Branches por feature → **Pull Request**.
- ◆ **Code review** obrigatório no PR.
- ◆ **CI** roda 190 testes a cada PR.

Gestão do board apoiada pelo **MCP do Jira** (ADR-0011): status e código sincronizados.

— DO REQUISITO AO CÓDIGO

Requisitos & rastreabilidade

8

épicas
EP00 – EP08

37

User Stories
MVP, todas Must

100%

rastreadas
RF · US · código

- ◆ Stories em **Gherkin** (Dado / Quando / Então), priorizadas por **MoSCoW**.
- ◆ Cada regra de negócio (RN09, RN10, RN16...) é **citada no código** que a implementa.



— COMO O CÓDIGO É ORGANIZADO

Arquitetura em camadas

- ◆ **Monólito** (ADR-0002), adequado ao escopo e ao prazo.
- ◆ **Ports & Adapters** (ADR-0007): controller → service → *interface* → repositório.
- ◆ **Domínio puro**, sem I/O: regras testáveis isoladamente.
- ◆ Aplicação **pragmática**: 2 adapters p/ a mesma porta provam a abstração; transações multi-tabela usam o ORM direto (exceção consciente).

```
HTTP → Controller → Service → IRepo (porta) → PgRepo  
(adapter) → Postgres  
|  
▼ domain/ (puro: status derivado,  
FEFO)
```

O CORAÇÃO DO SISTEMA

Status do pedido, derivado

- ◆ Um pedido é **cabeçalho + N itens**; cada item tem desfecho próprio (integral, parcial, falta).
- ◆ O status do pedido é **derivado** dos itens, nunca digitado à mão (RN10, ADR-0008).
- ◆ Máquina de estados de **7 regras por precedência**, em função pura testável.
- ◆ Consequência: o status **nunca fica inconsistente**, pois a verdade são os itens.

`domain/pedido.ts` → `statusDerivadoDoPedido()` ·
coberto por testes unitários.

— DECISÕES REGISTRADAS

Architecture Decision Records

ADR	Decisão	Alternativas rejeitadas
0005	Auth JWT + RBAC	sessão server-side; OAuth
0006	Drizzle ORM	Prisma; SQL cru; TypeORM
0007	Camadas / Ports & Adapters	controllers gordos; Active Record
0008	Pedido + status derivado	status manual; 1 pedido = 1 item
0010	Docker (4 containers)	VM direta; subdomínio; PaaS
0011	IA no fluxo de trabalho	uso ad-hoc; gestão manual

11 ADRs no total, cada uma com contexto, alternativas, consequências e link para o código.

Vamos ao sistema

Fluxo A — pedido ponta a ponta ·

Fluxo B — lotes & validade

Aplicação rodando localmente · stack Docker pronta para
deploy



— A REDE DE SEGURANÇA

Testes automatizados

190

testes
automatizados

137

backend
Vitest + Postgres

53

frontend
Vitest + RTL

- ◆ Backend testa contra **banco real** (não só mocks): cobre persistência.
- ◆ Domínio puro → regras críticas testadas em **milissegundos**.
- ◆ Tudo roda no **CI a cada PR**; verde é obrigatório para o merge.



— MAIS QUE TESTES

Revisão & refatoração

Dupla rede de revisão

- ◆ **Code review humano** no PR (autoridade final).
- ◆ **/code-review** assistido: lógica e diff.
- ◆ **Impeccable** (hook): design da UI.
- ◆ **CI**: testes + type-check.

Exemplos de refatoração

- ◆ Regra de status extraída para o **domínio puro** (reuso: criação + agente).
- ◆ Migration do EP08 **regenerada** via drizzle-kit, não editada à mão.
- ◆ Hardening da triagem com guards de entrada.

— COMO PUBLICAMOS

Infraestrutura & deploy

```
navegador → nginx (:80, /estoque-ho)
              |— frontend (SPA React)
              |— backend (Express :3000) → Postgres 16
[ rede interna do compose – só o nginx é exposto
]
```

- ◆ **Quatro containers** orquestrados por Docker Compose.
- ◆ Banco e backend **sem porta exposta**: superfície mínima de ataque.
- ◆ Subcaminho **/estoque-ho** no domínio do CIn: modular para outras clínicas.

Stack **pronta e validada localmente**; o deploy no servidor é essencialmente um comando.

O DIFERENCIAL

O agente de e-mail é **UX**, não tecnologia

- ◆ Solicitantes pedem ~uma vez por mês: obrigá-los a aprender um sistema é fricção alta.
- ◆ **Inversão do ônus**: o agente lê o e-mail que eles **já enviam** e cria o pedido como rascunho.
- ◆ O almoxarife **revisa** (triagem) em vez de digitar. Human-in-the-loop.
- ◆ Decisão de **experiência**, fundamentada no ADR-0004.

Backend e triagem prontos; a ingestão automática aguarda a caixa institucional.



Obrigado!

Perguntas?

Documentação completa ·

Grupo 5 · Desenvolvimento de Software · CIn-UFPE · 2026

Speaker notes